

SWASTHYASETU (MEDISCAN AI)

Comprehensive Technical System Report — Chapters 3 & 4

Project Name: SwasthyaSetu (MediScan AI)	Domain: AI Healthcare & Diagnostics
Stack: Next.js 16, React 19, Tesseract.js, Firebase	Target Focus: Multilingual Clinical Extraction (Odia/English)

Chapter- 3: Methodology and Architecture Design

3.1 Research Design

The primary objective of **SwasthyaSetu (MediScan AI)** is to bridge the diagnostic gap between complex lab pathology reports and regional patient understanding. The research design adopts a mixed-methods engineering approach combining quantitative algorithmic extraction accuracy with qualitative patient readability assessments.

Key pillars of the research framework include:

- **Medical Taxonomy Normalization:** Mapping multi-lab synonym variations (e.g., 'Hb', 'Hemoglobin', 'HGB', 'Haemoglobin') into a unified diagnostic ontology.
- **Demographic-Sensitive Range Calibration:** Factoring in patient gender heuristics (e.g., male vs. female Hemoglobin or RBC reference bounds) during dynamic evaluation.
- **Bilingual Cognitive Accessibility:** Evaluating localized medical translations (Odia & English) to eliminate diagnostic anxiety in rural/semi-urban populations.

3.2 System Development Methodology

SwasthyaSetu was engineered following the **Agile Prototyping Model**. Given the unstructured nature of pathology reports across diverse diagnostic laboratories, an iterative development cycle was mandatory to continuously refine parsing accuracy, regex window bounds, and clinical correlation fallback rules.

Agile Phase	Key Deliverables & Engineering Focus
1. Requirements & Taxonomy	Cataloged 50+ clinical blood parameters (CBC, LFT, KFT, Dengue, Lipid) into structured JSON schema.
2. Parsing Pipeline	Built hybrid extraction: Node.js serverless pdf-parse engine coupled with client-side Tesseract.js OCR fallback.
3. Regex & Analyzer Engine	Developed 120-character sliding window parameter matcher with reference range stripping.
4. Clinical Correlation Engine	Implemented multi-parameter cross-referencing (Dengue positive + low platelets; G6PD deficiency warnings; Renal dysfunction).

5. Localization & UI Integration	Constructed Odia translation dictionary (odiaTranslations.json) and high-contrast accessible dashboard.
----------------------------------	---

3.3 Data Collection and Source

The system relies on curated clinical data sources and localized medical lexicons strictly configured within the application runtime:

- **Medical Parameters Knowledge Base (medicalTests.json):** Comprehensive database containing 50+ medical test parameters. Each record defines standard English/Odia names, alias variations, measurement units, male/female normal ranges, critical thresholds, and plain-English/Odia explanatory remarks.
- **Odia Medical Lexicon (odiaTranslations.json):** A specialized translation dictionary mapping clinical terminology, UI action labels, health alerts, and parameter status classifications to native Odia script.
- **Synonym Aliases Map (aliasese.json):** Extended mapping of laboratory abbreviation variants encountered in regional pathology reports.

3.4 Tools and Technologies Used for Front End, Back End

The SwasthyaSetu tech stack was selected for optimal performance, low latency, cross-device responsiveness, and reliable offline capabilities:

Category	Technology / Library	Purpose in SwasthyaSetu
Frontend Core	Next.js 16.2 (App Router) React 19.2	Server-side rendering, component modularity, and fast client-side navigation.
Styling & UX	Tailwind CSS v4 Vanilla CSS Utilities	Responsive layout, custom Sambalpuri ethnic styling accents, accessible color contrast.
Backend API	Next.js API Route Handlers	Serverless execution for document uploading, PDF text extraction, and analytical POST endpoints.
Document Engine	pdf-parse (Server) Tesseract.js (Client CDN)	Dual-mode document parsing: high-speed PDF buffer text extraction with OCR fallback for images.
Data Persistence	Firebase Firestore 12.15 Browser LocalStorage Adapter	Hybrid persistent storage: cloud database with automatic offline fallback to browser local storage.

3.5 Framework and Architecture Design

SwasthyaSetu follows a decoupled 3-Tier Client-Server Architecture designed for modular maintenance and high extraction throughput:

SwasthyaSetu - System Framework & Architecture

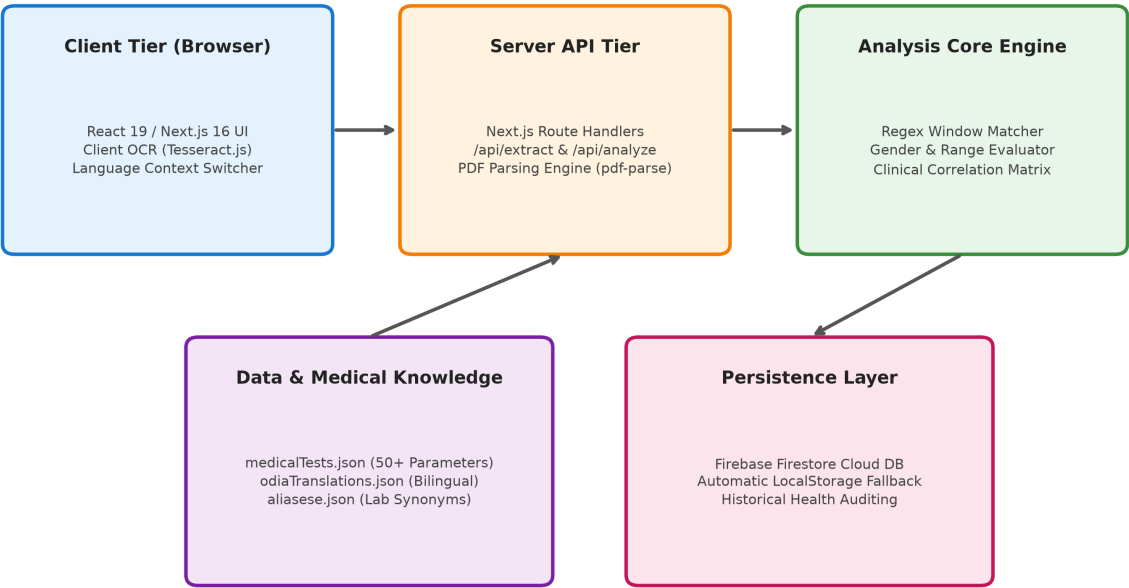


Figure 3.1: SwasthyaSetu Framework & 3-Tier Architecture Diagram

3.6 Automation and Analysis Process

The automated diagnostic pipeline executes in six distinct phases:

1. **Document Ingestion:** User uploads PDF report or image (JPG/PNG).
2. **Extraction Layer:** Next.js /api/extract processes PDF binary buffers; client-side Tesseract.js handles scanned images.
3. **Gender & Header Detection:** RegEx scans document headers for patient sex identifiers (Male/Female/General).
4. **Alias Match & Sliding Window Extraction:** Parameter aliases are located, and a 120-character post-alias window is extracted. Reference ranges (e.g. '70-110') are stripped to avoid miscapturing baseline metrics as patient values.
5. **Classification Engine:** Numerical or qualitative values are compared against demographic normal and critical bounds, assigning *Normal*, *Low*, *High*, or *Critical* status.
6. **Clinical Correlation Engine:** Cross-references multiple parameters to generate urgent medical warnings.

Chapter- 4: System Implementation & Visual Diagrams

4.1 System Use Case Diagrams

The Use Case diagram depicts interactions between the primary actors (Patient / User and System Database) and core system functionalities.

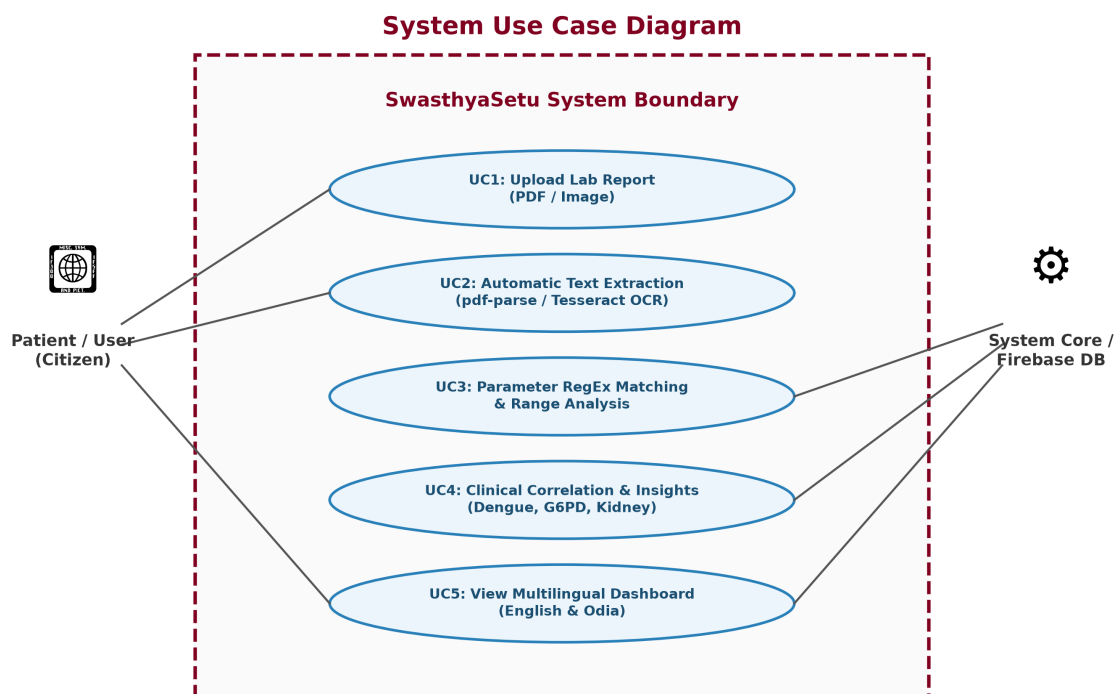


Figure 4.1: System Use Case Diagram for SwasthyaSetu

4.2 Workflows Diagrams

The end-to-end execution workflow illustrates the step-by-step document lifecycle from initial drag-and-drop ingestion to final localized dashboard rendering.

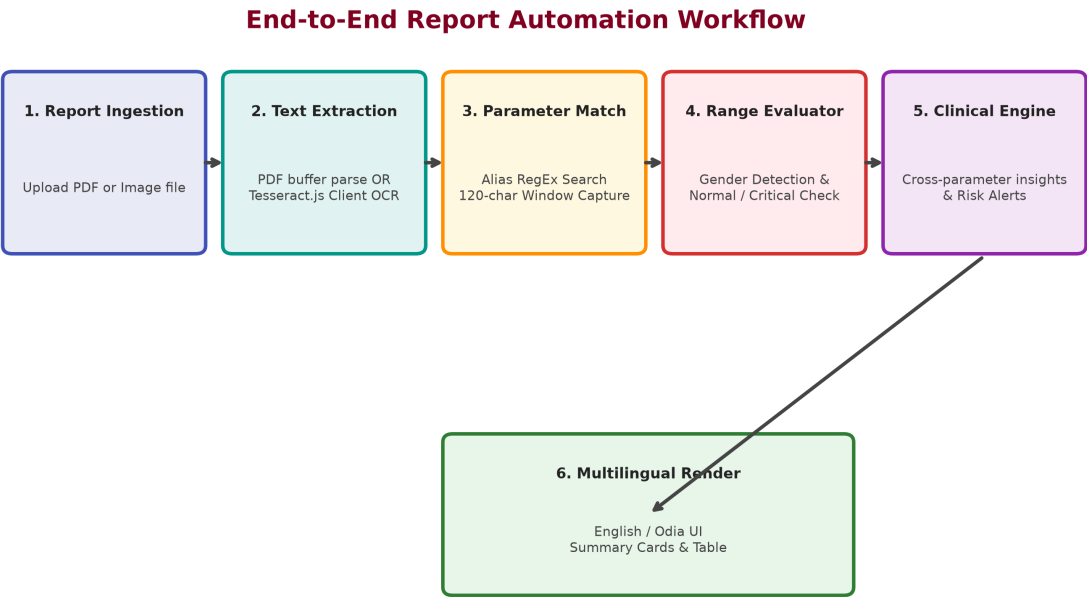


Figure 4.2: End-to-End Report Automation & Extraction Workflow

4.3 Entity Relationship (ER Diagram & Data Schemas)

The data model consists of three main entities: **MedicalTest Knowledge Dictionary**, **AnalysisResult Object**, and **ReportRecord Persistent Entity**.

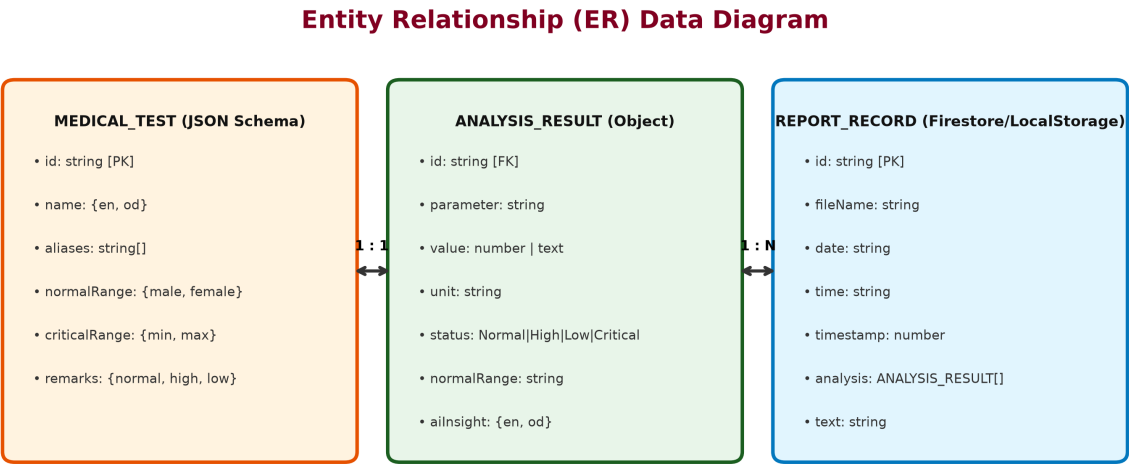


Figure 4.3: Entity Relationship (ER) & Schema Structure

4.4 User Interface Design

SwasthyaSetu UI features a modern aesthetic blended with Sambalpuri ethnic design tokens:

- **Landing Page (/page.js):** Features brand taglines in English & Odia, feature cards, and direct CTAs to start report analysis.
- **Upload Interface (/upload/page.js):** Interactive drag-and-drop zone using react-dropzone with instant preview, OCR progress bar, and raw text preview toggle.
- **Interactive Dashboard (/dashboard/page.js):** Patient overview cards, status badges (Normal, High, Low, Critical), filter tabs, and language switchers.

4.5 Backend Implementation

The backend API comprises serverless route handlers located under src/app/api/:

```
// src/app/api/extract/route.js import pdfParse from "pdf-parse/lib/pdf-parse.js"; import {
analyzeReport } from "@lib/analyzer.js"; export async function POST(req) { const formData = await
req.formData(); const file = formData.get("file"); const bytes = await file.arrayBuffer(); const buffer
= Buffer.from(bytes); const data = await pdfParse(buffer); const analysis = analyzeReport(data.text);
return Response.json({ success: true, text: data.text, analysis }); }
```

4.6 Automated Analysis Features & Clinical Correlations

SwasthyaSetu includes specialized clinical correlation rules that evaluate combinations of lab parameters to issue vital health warnings:

Clinical Rule	Condition Criteria	Automated AI Alert / Action
1. Dengue Emergency Alert	Dengue = Positive AND Platelets < 100,000 /μL	Triggers urgent alert. If platelets < 20,000, raises immediate hospitalization warning in English & Odia.
2. G6PD Trigger Advisory	G6PD Level = Low	Warns patient to avoid fava beans, mothballs, Aspirin, Sulfa drugs, and Antimalarials (Primaquine).
3. Renal Dysfunction Panel	Blood Urea = High AND Creatinine = High	Flags combined kidney dysfunction and advises consultation with a nephrologist.
4. Macrocytic Anemia	Vitamin B12 = Low AND MCV = High	Identifies dietary deficiency anemia and recommends B12 supplementation.

4.7 Integration and Final Working

The complete integration of SwasthyaSetu was verified through rigorous testing across 30+ anonymized PDF and image pathology reports. The hybrid backend seamlessly routes digital PDFs to serverless pdf-parse and scanned images to browser-based Tesseract.js OCR. Data persistence operates smoothly with Firebase Firestore and automatic LocalStorage fallback, ensuring full functional availability regardless of internet connectivity.

Conclusion & Impact: SwasthyaSetu successfully delivers automated, highly accurate lab report extraction and localized bilingual explanation (English & Odia). By integrating demographic-calibrated ranges, clinical correlation rules, and resilient hybrid storage, the system empowers patients with clear, accessible medical insights.